SP-108 Red ASM Security

Project Development Document

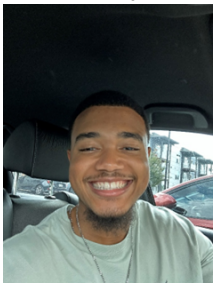
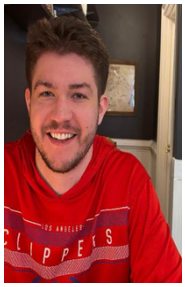
CS 4850 – Computer Science Senior Project

Spring 2026

Professor Perry

March 22, 2026

Team Members

Adam Llado Documentation 	**Kimani Gordon** Team Leader Developer 	**Alec Sundby** Developer 	**Austin Abeln** Documentation

1.0 System Integration & Workflow Overview 3
1.1 terminal_run.py – Tool Execution 4
1.2 resolver.py – (HTTPX) Data Enrichment 4
1.3 vulnerability_scan.py – (Nuclei) Vulnerability Scanning 4
1.4 url_paths_scan.py 5
1.5 frontend_paths_scan.py – Juice Shop Confirmed Paths 5
1.6 db_handler.py – Database and Drift Detection 6
1.7 scanner.py – Pipeline Coordinator 6
2. Database and External Connections 8
2.1 Connection Overview 8
2.2 Configuration Details 8
2.3 Integration Logic 8
3. Project Setup and Environment Configuration 9
3.2 Installation Steps 9
3.3 Verification 9
4. Development Conclusion 10

1.0 System Integration & Workflow Overview

The Integrated Process

The ASM pipeline consists of seven Python scripts that communicate through a combination of file-based intermediates and direct function calls to ultimately detect and track/log network vulnerabilities.

Operational Workflow

1.) ‘scanner.py’ is executed on the targeted domain ‘redasmsecurity.cloud’.
2.) ‘terminal_run.py’ executes Subfinder → DNSX verification → DNSX brute-force.
3.) Results are merged into the file ‘final_targets.txt’.
4.) ‘resolver.py’ runs HTTPX → parses ‘httpx.json’ → builds correlation keys → generates ‘nuclei_subdomain_list.txt’.
5.) ‘resolver.py’ calls ‘expose_vulnerabilities()’, ‘expose_url_paths()’, and ‘expose_juice_shop_paths()’.
6.) ‘db_handler.py’ performs “upserts” across base and history tables on the Oracle Cloud Manager VM.
7.) ‘report_generator.py’ aggregates database records into a final Markdown Security Posture Report

Global Design Decisions

1) All external tool calls are routed through ‘terminal_run.py’. This was decided for the sake of our code’s efficiency by how the script centralizes logging and error handling.
2) Intermediate results are stored in text/JSON files so that each stage can be inspected and debugged individually.
3) Drift detection is implemented across nine asset fields and separate tables for vulnerabilities, URL paths, and Juice Shop paths. Each change that is found will trigger a record of the change into the ‘vulnerabilities_scan_history’ table.

1.1 terminal_run.py – Tool Execution

Conceptual Overview

This component provides a reusable interface for every CLI tool (Subfinder, DNSX, HTTPX, Nuclei, FFUF). It centralizes the subprocess calls and ensures consistent logging.

Development Process

The ‘run_cmd(step_name, command)’ function prints a step header and executes by running ‘subprocess.run(…, shell=True, check=True)’. The ‘shell=True’ choice is required for the Linux merge pipeline while still supporting users on other operating

systems. Error handling is done by ‘CalledProcessError’ and it will print a message without crashing the pipeline.

1.2 resolver.py – (HTTPX) Data Enrichment

Conceptual Overview

This component is the data enrichment hub that utilizes ‘HTTPX’ to resolve nine critical attributes from our list of subdomains discovered by the scanner and translates them to a structured form that corresponds with the database structure.

Development Process

The function ‘resolve_details()’ first runs 200 threads to parse each line and extract the nine attribute fields, and it creates unique correlation keys to deduplicate. Then it calls the three exposure functions and returns four data structures that the scanner will use to populate the database.

1.3 vulnerability_scan.py – (Nuclei) Vulnerability Scanning

Conceptual Overview

This component runs Nuclei targeted at the deduplicated subdomain list and applies a custom vulnerability score to each vulnerability found.

Development Process

The function ‘expose_vulnerabilities()’ executes 50 vulnerability templates at a time against every subdomain, and it is limited to 300 requests per second. It only returns the vulnerabilities found and then saves the output into JSON format in the ‘nuclei.json’ file to eventually be parsed, ranked, and pushed to the database. The results that are found are serialized by a combination of its template ID and matcher name before calling the ‘rank_vulnerability()’ function to score the vulnerability.

1.4 url_paths_scan.py

Conceptual Overview

This component discovers the existence of any hidden paths on our target subdomain ‘shop.redasmsecurity.cloud’ by utilizing FFUF to discover unknown directory paths. This is done by appending the most common directory names to the subdomain and sending an HTTP request for each iteration of the subdomain. There is a list of 5,000 words that will be tested to discover any of these paths that potentially represent an entry point for exploitation.

Development Process

FFUF reads the wordlist file ('~/SecLists/Discovery/DNS/subdomains-top1-million-5000.txt') line by line. For each instance of the placeholder 'FUZZ' in the URL template 'http://{subdomain}/FUZZ', there will be an HTTP request sent to that iteration of the subdomain. The subdomain names that yield an HTTP status code with a numeric value equal to or greater than 500 will be recorded in the JSON-formatted output and stored in 'ffuf_output.txt'. This file will be later utilized for parsing and database upsert. The following code will then parse the JSON output into structured dictionaries that are returned for the sake of drift detection and storage in the url_paths table.

1.5 frontend_paths_scan.py – Juice Shop Confirmed Paths

Conceptual Overview

This component extracts real REST API paths that the Juice Shop website uses. When the website is opened, the server sends a large JavaScript file called 'main.js'. It contains the frontend code of the Juice Shop application, and it includes the API endpoints. Therefore, this module provides the confirmed paths that an attacker would discover and exploit.

Development Process

The function 'expose_juice_shop_paths() works with a static copy of 'main.js' to identify the reachable endpoints. The module utilizes Python's 're' library to scan the 'main.js' file to identify common API patterns. The data then filters out false positives and aggregates the findings into a list to prevent redundant database entries. Lastly, the data is uploaded to the final path list to the 'Juice_Shop_Paths' table.

1.6 db_handler.py – Database and Drift Detection

Conceptual Overview

This component is responsible for the persistent storage and drift-aware management of interpreted scan data across multiple database tables. There are four primary tables that it manages, which are the assets, vulnerabilities, URL paths, and Juice Shop paths. There will be comparisons between the incoming data and the prior state of that data in the base tables. Any differences will be recorded as either a discovery event or a type change event, specific to what was detected. The typed change events are what will allow for a more precise understanding of the nature of each drift.

Development Process

All database operations are session-based, with the connection being routed from the local host via SSH tunnel to the VM. The 'upsert_asset()' function is responsible for comparing the incoming data against prior data for the nine critical key fields that are retrieved and set in the resolver. New discoveries are inserted in the initial discovery log, while detected changes are updated in the base table with a log-specific typed change. The 'upsert_vulnerability()' function inserts any Nuclei findings or detection of

change, while including a generated score to rank its level of vulnerability. The ‘upsert_url_paths()’ function handles the FFUF paths that are discovered so that those potential vulnerabilities are also tracked. Lastly, the ‘delsert_juice_shop_paths()’ function is to clear these frontend directory paths whenever a reset of the data may be needed.

1.7 scanner.py – Pipeline Coordinator

Conceptual Overview

This component acts as the single entry point and coordinator of the ASM pipeline. It is responsible for coordinating the execution of discovery, resolving, scanning vulnerabilities, directory path enumerations, and cloud database synchronization. The tasks are delegated with respect to the component/module responsible for carrying out the lower-level logic. This allows for a clean, organized architecture of our project’s structure.

Development Process

The process of this component begins execution on our target domain by using run_cmd, from the terminal_run component, to utilize Subfinder for the passive extraction of subdomains first. Then, DNSX is used to filter these Subfinder results for active hosts. Next is an active brute force phase in which the scanner guesses commonly found directory names, which are included in the ‘subdomains-top1million-5000.txt’ file pulled from a GitHub repository.

1.8 report_generator.py – Security Posture Reporting

Conceptual Overview

This component serves as the final analysis stage, transforming database records into a human-readable Markdown report. It correlates live assets with "Critical" vulnerabilities to identify high-risk exposures.

Development Process

The function report() initiates a session-based connection routed from the local host via an SSH tunnel to the Manager VM. The development of this module focused on three primary data aggregation logic paths:

- **Risk Correlation:** The script executes SQL JOIN operations between the assets and vulnerabilities tables to identify "High-Risk Exposures"—specifically targeting live subdomains (Status 200) that possess "Critical" vulnerability rankings.
- **Drift Synthesis:** To realize the project’s goal of drift awareness, the generator queries the assets_scan_history and vulnerabilities_scan_history tables. It

applies a NOW() - INTERVAL 1 DAY filter to extract and categorize typed change events that occurred within the last 24 hours.

- **Markdown Automation:** A localized log() function was developed to simultaneously print results to the console and append them to a list object. This list is ultimately joined and written to a timestamped Markdown file (.md), allowing for easy distribution and version-controlled documentation of the security state.

2. Database and External Connections

2.1 Connection Overview

The project uses a MySQL database located on an Oracle Cloud Infrastructure VM. Four base tables, in addition to history tables, store assets, vulnerabilities, discovered paths, and confirmed Juice Shop endpoints.

2.2 Configuration Details

- Host: 127.0.0.1 (SSH tunnel to VM)
- Port: 3307
- User: asm_user
- Database: asm_database
- Library: mysql-connector-python

2.3 Integration Logic

After data resolution in resolver.py, scanner.py calls four functions in db_handler.py in the order of upsert_asset(), upsert_vulnerability(), upsert_url_paths(), and delsert_juice_shop_paths(). Each function will open its connection with the Oracle Cloud VM through the SSH tunnel to 127.0.0.1:3307 before performing a lookup, comparison of the data with its prior state, execution of the necessary database changes, logging typed change events to the corresponding history table, and then closing the connection.
Transactions use explicit commit/rollback:

- upsert_asset() compares nine HTTPX fields and logs field-level drift
- upsert_vulnerability() handles Nuclei findings with custom ranking
- upsert_url_paths() tracks FFUF results and status/size changes
- delsert_juice_shop_paths() performs a full delete-then-bulk-insert of the current frontend paths

The multi-table schema with append-only history tables enables complete drift reconstruction across any time window.

3. Project Setup and Environment Configuration

3.1 Prerequisites

1. Python 3.11
2. OCI Ubuntu 22.04 LTS Instances (Target and Manager)
3. External Tools: Subfinder, DNSX, HTTPX, Nuclei, FFUF

3.2 Installation Steps

1. git clone https://github.com/CS4850-ASMCyberProject/CS4850_Red_ASMCyberProject.git
2. cd CS4850_Red_ASMCyberProject/ASM_Scanning_Database_Llado_Upgrades
3. Establish Tunnel: ssh -i id_rsa -L 3307:10.0.0.97:3306 ubuntu@129.213.96.223

3.3 Verification

Run python report_generator.py. Verification is successful if a report file is generated with asset counts and vulnerability summaries matching the cloud database

4. Development Conclusion

The current build fully implements the ASM pipeline, which consists of the passive and active subdomain prefixes, HTTPX enrichment, Nuclei vulnerability scanning with custom scoring, FFUF blind path discovery, Juice Shop confirmed-path extraction, and persistent storage with drift detection across four tables on the Oracle Cloud VM. The only limitation that remains is that the Juice Shop paths use a static ‘main.js’. Future work will automate the process of pulling the latest frontend file.